TeamConnect — Implementation Plan

> **Stack**: React + Vite + Tailwind + Framer Motion | Node.js + Express + Prisma (SQLite) + Socket.io | JWT Auth

Project Structure

```
c:\.vscode\CEP/
├── client/                          # React + Vite frontend
│   ├── public/
│   │   ├── manifest.json            # PWA manifest
│   │   └── sw.js                    # Service worker
│   ├── src/
│   │   ├── assets/
│   │   ├── components/
│   │   │   ├── ui/                  # Reusable: Button, Card, Badge, Input, Modal, Toast...
│   │   │   ├── cursor/              # CustomCursor, SpotlightGlow, ParticleTrail
│   │   │   ├── layout/              # Navbar, Sidebar, Footer, PageTransition
│   │   │   ├── matching/            # MatchScoreBadge, GapAnalysisChart, WhatIfSimulator
│   │   │   ├── projects/            # ProjectCard, RoleCard, RoleBuilder, KanbanBoard
│   │   │   ├── profile/             # ProfileStrength, SkillBadge, Timeline
│   │   │   ├── notifications/       # NotificationBell, NotificationItem
│   │   │   └── CommandPalette.jsx
│   │   ├── pages/
│   │   │   ├── public/              # Landing, StudentLogin, StudentRegister, AdminLogin
│   │   │   ├── student/             # Dashboard, ProjectDetail, CreateProject, MyProjects,
│   │   │   │                        # TeamWorkspace, IdeaBoard, Profile, Notifications,
Onboarding
│   │   │   └── admin/               # AdminDashboard, ManageProjects, ManageStudents,
│   │   │                            # ManageSkills, Challenges, Recommendations, AuditLog,
Reports
│   │   ├── hooks/                   # useAuth, useSocket, useMatchScore, useCommandPalette
│   │   ├── context/                 # AuthContext, SocketContext, ThemeContext
│   │   ├── lib/                     # api.js (axios), matchingEngine.js, utils.js
│   │   ├── router/                  # Routes, ProtectedRoute, AdminRoute
│   │   └── main.jsx
│   ├── tailwind.config.js
│   ├── vite.config.js
│   └── package.json
│
├── server/                          # Node.js + Express backend
│   ├── prisma/
│   │   ├── schema.prisma            # All models
│   │   ├── seed.js                  # Realistic demo data
│   │   └── migrations/
│   ├── src/
│   │   ├── controllers/             # auth, projects, roles, applications, teams,
│   │   │                            # ideas, challenges, recommendations, admin, users,
matching
```

```
│   │   ├── middleware/           # auth.js, roleCheck.js, institutionIsolation.js,
rateLimit.js
│   │   ├── routes/               # auth, projects, roles, applications, teams,
│   │   │                         # ideas, users, admin, matching, recommendations
│   │   ├── services/
│   │   │   ├── matchingEngine.js  # Core matching algorithm (unit-tested)
│   │   │   ├── emailService.js    # Mocked email / weekly digest
│   │   │   ├── certService.js     # PDF certificate generation
│   │   │   └── moderationService.js # Profanity filter
│   │   ├── socket/               # Socket.io handlers (chat, notifications)
│   │   └── app.js
│   ├── .env.example
│   └── package.json
│
├── README.md
└── .env.example
```

Build Phases

Phase 1 — Foundation

- Scaffold client (Vite + Tailwind + Framer Motion + React Router)
- Scaffold server (Express + Prisma + JWT + bcrypt)
- Define all Prisma models
- Auth: register, login, JWT middleware
- Institution isolation middleware

Phase 2 — Onboarding & Profile

- Onboarding wizard (skills, proficiency, interests, bio, links, availability)
- Profile strength progress bar
- Student profile page + public shareable view

Phase 3 — Projects & Roles

- Create project with role builder
- Discover Projects page with match scores
- Project Detail page with gap analysis radar chart

Phase 4 — Matching Engine

- Backend matching service (skill coverage 60%, interests 15%, credibility 15%, availability 10%)
- Skill similarity map
- "Why you match" + "Skills missing" explanations
- "Close the gap" learning resources
- "What-if" simulator

Phase 5 — Applications & Teams

- Apply to roles
- Owner accepts/rejects (ranked applicants)
- Team Workspace: members, Kanban, milestones, announcements
- Suggested students per role (ranked)

Phase 6 — Real-time Chat

- Socket.io integration
- Team chat room
- Direct messaging with applicants

Phase 7 — Idea Board & Challenges

- Post raw ideas, upvote, comment
- "Pick this up" → create project
- Community challenges (admin-posted)

Phase 8 — Teacher Recommendations

- Student requests recommendation
- Unique secure link for teacher
- Admin moderation queue

Phase 9 — Badges, Peer Reviews, Certificates

- Badge system (auto-awarded)
- Peer review form after project completion
- PDF certificate with QR verification code

Phase 10 — Admin Panel

- Dashboard with charts (Recharts)
- Skill supply vs demand heatmap
- Project moderation, student management
- CSV/PDF export

Phase 11 — UX Polish

- Ctrl+K command palette
- PWA manifest + service worker
- Skeleton loaders, toasts, confetti

Phase 12 — Visual & Cursor Polish

- Dark theme (aurora gradient mesh)
- Glassmorphism cards
- Custom cursor: dot + trailing ring
- Mouse spotlight glow
- Canvas particle trail
- 3D card tilt + light glare
- Magnetic buttons
- Hero node-network animation
- Scroll-reveal animations

Seed Data (Demo Credentials)

Institutions

1. **MIT Institute of Technology** — domain: `mit.edu`
2. **Stanford University** — domain: `stanford.edu`

Students (12 total, 6 per institution)

- Student demo: `alice@mit.edu` / `demo1234`

Admin

- Admin demo: `admin@mit.edu` / `admin1234`

Projects (10), Ideas (4), Challenges (2), Teachers (3)

Key Design Decisions

- SQLite for dev (Prisma makes it trivial to switch to PostgreSQL)
- JWT stored in httpOnly cookies for security
- Institution isolation enforced at Prisma query level (institutionId filter on every query)
- Matching engine is pure function: `computeMatchScore(student, role, project)` → easily unit-testable
- Socket.io rooms are namespaced by `institution:project` to enforce isolation
- Profanity filtering via `bad-words` npm package
- PDF generation via `pdfkit`
- QR codes via `qrcode` npm package